Text Vectorization and Word Embeddings

Sourav Karmakar

Outline

Foundations of Text Vectorization

The Need for Vectorization

Machine learning algorithms and deep neural networks are fundamentally mathematical functions.

- They expect **numerical inputs** (vectors, matrices, tensors).
- They cannot directly process raw text (strings).

What is Vectorization?

The process of converting unstructured text into a structured, numerical format that algorithms can process. An ideal vectorization scheme captures not just the word, but its underlying **semantic meaning**.

Essential NLP Terminology Review

Before mathematically representing text, we must define our dimensional space.

- **Corpus:** A large, structured dataset of text documents used for training.
- **Document:** A single piece of text within the corpus (e.g., a review, a sentence).
- **Tokenization:** The process of breaking a document into its fundamental units (Tokens).
- **Vocabulary (V):** The set of all unique tokens present in a corpus.

Vocabulary Size

The size of the vocabulary, denoted as $|V|$, dictates the mathematical dimensionality of many traditional vectorization techniques.

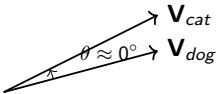
Measuring Similarity in Vector Space

Once text is represented mathematically as vectors, we measure similarity using **Cosine Similarity**.

Instead of calculating Euclidean distance, we measure the cosine of the angle (θ) between two vectors. This measures *orientation*.

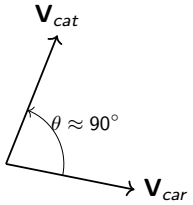
$$\text{Cosine Similarity}(\mathbf{A}, \mathbf{B}) = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} \tag{1}$$

High Similarity (Small Angle)



$\cos(\theta) \approx 1$

Low Similarity (Orthogonal)



$\cos(\theta) \approx 0$

Traditional Count-Based Vectorization

One-Hot Encoding (OHE)

The most straightforward vectorization method. Each unique word in V is represented by a binary vector of dimension $|V|$.

If word w_i is the i-th word in the vocabulary, its vector $\mathbf{v}_{w_i}$ has a 1 at index i, and 0 everywhere else.

Example: Sentence = "The cat sat" $\rightarrow V = \{\text{The, cat, sat}\}, |V| = 3$.

$$\mathbf{V}_{\text{The}} = [1, 0, 0]$$
$$\mathbf{V}_{\text{cat}} = [0, 1, 0]$$
$$\mathbf{V}_{\text{sat}} = [0, 0, 1]$$

Mathematical Property: Orthogonality. The dot product of any two distinct words is always 0 ($\mathbf{V}_{\text{cat}} \cdot \mathbf{V}_{\text{sat}} = [0, 1, 0] \cdot [0, 0, 1] = 0$). This implies *zero* similarity!

Count Vectorization (Bag-of-Words)

Moving from word-level to document-level representations.

- We represent a document by the **frequency** of each vocabulary word it contains.
- Disregards grammar and word order ("bag" of words).
- Creates a **Document-Term Matrix M** of size $D \times |V|$.

Example:
D_1: "The cat sat on the mat"
D_2: "The dog sat on the log"
$V = [\text{The}, \text{cat}, \text{sat}, \text{on}, \text{mat}, \text{dog}, \text{log}], |V| = 7$

	The	**cat**	**sat**	**on**	**mat**	**dog**	**log**
Doc 1	2	1	1	1	1	0	0
Doc 2	2	0	1	1	0	1	1

TF-IDF Vectorization

Count Vectorization is biased towards highly common words (like "the"). **TF-IDF** assigns a weight that is high if a word is frequent in a document, but *rare* across the whole corpus.

1. Term Frequency (TF):

$$TF(t, d) = \frac{\text{count of } t \text{ in } d}{\text{total terms in } d} \tag{2}$$

2. Inverse Document Frequency (IDF):

$$IDF(t, D) = \log\left(\frac{1 + N}{1 + df_t}\right) + 1 \tag{3}$$

Where N is total documents, df_t is number of documents containing term t.

Final Score

$$\text{TF-IDF}(t, d) = TF(t, d) \times IDF(t, D)$$

Numerical Example: TF-IDF Setup

Corpus (3 documents):

Doc	Text
d_1	the cat sat on the mat
d_2	the dog sat on the log
d_3	the cat chased the dog

Compute TF-IDF for term $t =$ "cat" in d_1:

Step 1 — Term Frequency:

$$TF(\text{cat}, d_1) = \frac{\text{count of "cat" in } d_1}{\text{total terms in } d_1} = \frac{1}{6} \approx 0.167$$

Numerical Example: TF-IDF Score

Step 2 — Inverse Document Frequency:

$N = 3$ (total docs), $df_{\text{cat}} = 2$ ("cat" appears in d_1 and d_3)

$$IDF(\text{cat}, D) = \log\left(\frac{1+3}{1+2}\right) + 1 = \log\left(\frac{4}{3}\right) + 1 \approx 0.288 + 1 = 1.288$$

- $TF(\text{cat}, d_1) \approx 0.167$
- $IDF(\text{cat}, D) \approx 1.288$

Step 3 — Final TF-IDF Score:

$$\text{TF-IDF}(\text{cat}, d_1) = 0.167 \times 1.288 \approx \mathbf{0.215}$$

TF-IDF: Document-Term Matrix

Applying TF-IDF across **all words** and **all documents** produces a document-term matrix, where each cell is the TF-IDF score of that word in that document.

	the	**cat**	**sat**	**on**	**mat**	**dog**	**log**	**chased**
d_1	0.333	0.215	0.215	0.215	0.282	0	0	0
d_2	0.333	0	0.215	0.215	0	0.215	0.282	0
d_3	0.400	0.258	0	0	0	0.258	0	0.339

The document-term matrix is the **final feature matrix** passed to the Machine Learning Model. Each row is one training example; each column is one feature (word).

Limitations of Traditional Methods

While count-based baselines are highly interpretable, they suffer from severe mathematical and semantic flaws:

1. **Sparsity & Dimensionality:** A corpus with 50,000 unique words results in vectors with 50,000 dimensions, where 99% of the values are zeros.
2. **Loss of Word Order:** "Cat sat on dog" and "Dog sat on cat" produce the exact same Bag-of-Words vector.
3. **No Semantic Understanding:** OHE treats "car" and "automobile" as completely unrelated orthogonal vectors. The model has no concept of synonymy.

Introduction to Word Embeddings & Word2Vec

The Distributional Hypothesis

To capture semantic meaning, we move to **Neural Network-based Predictive Models**. The foundation of this field is the Distributional Hypothesis:

> *"A word is characterized by the company it keeps."*

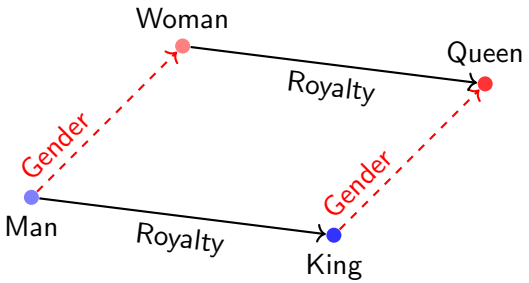
If we train a neural network to successfully predict a word based on its surrounding context words, the hidden layer weights of that network will organically learn a dense vector representation containing the word's semantic meaning.

Semantic Vector Arithmetic

The most remarkable property of Word2Vec embeddings is capturing complex semantic relationships via linear vector translations.

The Classic Analogy: "King is to Queen as Man is to Woman".

$$\mathbf{V}_{\text{king}} - \mathbf{V}_{\text{man}} + \mathbf{V}_{\text{woman}} \approx \mathbf{V}_{\text{queen}} \tag{4}$$

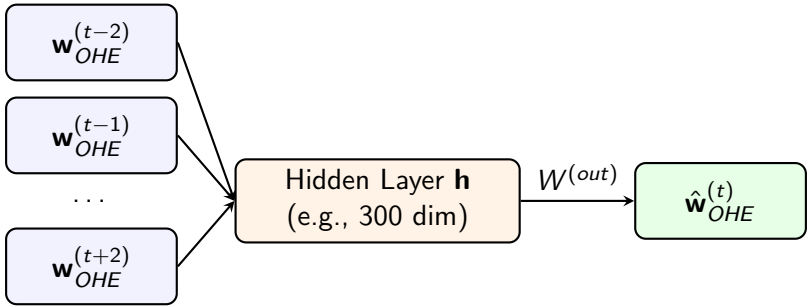


Word2Vec Architectures: Deep Dive

Continuous Bag-of-Words (CBOW)

CBOW tries to predict a single target word from its surrounding context words within a window size k.

Example ($k = 2$): "There is a lot of [**advancement**] in AI recently".
Target: *advancement*. Context: *a*, *lot*, *in*, *AI*.



CBOW Mathematical Formulation

Let's trace the math through the shallow neural network.

Step 1: Input to Hidden Layer

The one-hot context vectors are multiplied by an input weight matrix $\mathbf{W}_{h\times|V|}^{(in)}$.

$$\vec{h}_{t-2} = \mathbf{W}^{(in)} \cdot \mathbf{w}_{OHE}^{(t-2)} \tag{5}$$

Step 2: Averaging the Context

The hidden layer averages these projections:

$$\vec{h} = \frac{1}{4}\left(\vec{h}_{t-2} + \vec{h}_{t-1} + \vec{h}_{t+1} + \vec{h}_{t+2}\right) \tag{6}$$

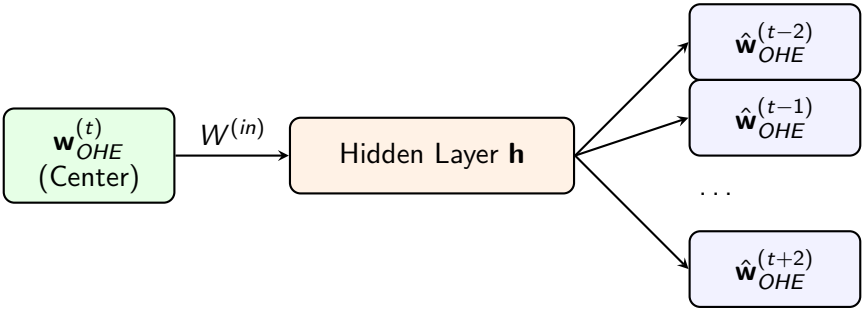
Step 3: Output Projection

The hidden representation is projected back to $|V|$ dimensions via $\mathbf{W}_{|V|\times h}^{(out)}$ and passed through a Softmax function to generate probabilities.

$$\hat{\mathbf{w}}_{OHE}^{(t)} = \text{Softmax}\left(\mathbf{W}^{(out)} \cdot \vec{h}\right) \tag{7}$$

Skip-gram Architecture

Skip-gram flips the problem: It uses a single center target word to predict the surrounding context words.



Why Skip-gram? While computationally more expensive than CBOW, Skip-gram is renowned for generating higher quality embeddings, especially for infrequent words.

Extracting the Learnt Word Vectors

Once the network finishes training, we discard the prediction task. **The weights of the neural network become our word embeddings.**

Specifically, we extract the **Input Weight Matrix** $\mathbf{W}^{(in)}_{h \times |V|}$.

$$\mathbf{W}^{(in)} = \left[\begin{array}{c|c|c|c|c} w_1 & w_2 & \dots & \mathbf{w}_{\text{advance}} & \dots \\ \hline 0.1 & -0.2 & \dots & \mathbf{0.01} & \dots \\ -0.4 & 0.8 & \dots & \mathbf{0.23} & \dots \\ \vdots & \vdots & \ddots & \vdots & \ddots \\ 0.9 & 0.1 & \dots & \mathbf{-0.71} & \dots \end{array}\right]$$

Each column corresponds to a specific word. Because we set $h = 300$, extracting the column for "advancement" gives us a 300-dimensional dense semantic vector!

Making Word2Vec Training Feasible (Optimization)

The Softmax Bottleneck

The standard Softmax function is computationally disastrous for NLP.

$$P(w_j|w_t) = \frac{\exp(\mathbf{v'}_{w_j} \cdot \mathbf{v}_{w_t})}{\sum_{i=1}^{|V|} \exp(\mathbf{v'}_{w_i} \cdot \mathbf{v}_{w_t})} \tag{8}$$

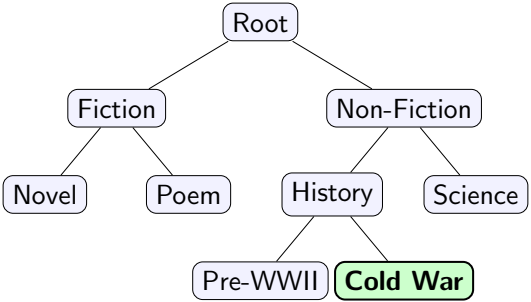
The Problem: For every single training step, calculating the denominator requires summing exponentiated vector products over the **entire vocabulary** $|V|$ (often 50,000+ words).

Solution: We replace the standard Softmax with optimization techniques:

1. Hierarchical Softmax
2. Negative Sampling

Optimization 1: Hierarchical Softmax

Reframes the $|V|$-class prediction into a series of binary classifications by arranging the vocabulary as leaves in a binary **Huffman Tree**.



The probability of predicting "Cold War" is the product of binary decisions down the path: $P(\text{path}) = P(\text{Right})P(\text{Left})P(\text{Right})$.

Complexity reduces from $O(|V|)$ to $O(\log_2 |V|)$.

Optimization 2: Negative Sampling

Instead of calculating probabilities for all words, we treat the problem as **Logistic Regression**.

For a center word (e.g., "quick") and its true context word ("brown"), we generate:

- 1 **Positive Sample**: ("quick", "brown") $\rightarrow$ Target $= 1$
- k **Negative Samples**: Random words from the vocabulary.
  e.g., ("quick", "apple") $\rightarrow$ Target $= 0$; ("quick", "sky") $\rightarrow$ Target $= 0$.

Rules of Thumb:

- $k = 2 - 5$ for large corpora.
- $k = 5 - 10$ for smaller corpora.

Note: Negative words are sampled using a unigram distribution raised to the 3/4 power, $u(w)^{3/4}$, to dampen the frequency of highly common words like "the".

Limitations of Word2Vec

Despite its revolutionary impact, Word2Vec has notable limitations:

1. **Out-of-Vocabulary (OOV) Words:**
   Word2Vec maps entire words to vectors. If a user types a misspelling or a new slang word not present in the training vocabulary, the model crashes; it cannot deduce a vector on the fly.

2. **Strictly Local Context Window:**
   Because the algorithm only scans small sliding windows (e.g., ± 2 words), it completely fails to capture the global, macro-statistics of the entire document.

Beyond Word2Vec: GloVe and FastText

(Addressing Word2Vec Limitations)

Addressing Local Context: GloVe

GloVe (Global Vectors for Word Representation) was developed by Stanford researchers to address Word2Vec's "local window" limitation.

- **How it works:** Instead of relying purely on a predictive sliding window, GloVe constructs a massive global word-word co-occurrence matrix for the entire corpus.
- **The Math:** It utilizes matrix factorization techniques (similar to LSA) combined with local context windows.
- **The Result:** Embeddings that capture the semantic richness of Word2Vec, but are firmly grounded in the global statistical properties of the corpus.

Addressing OOV: FastText

FastText was developed by Facebook's AI Research lab to solve the Out-Of-Vocabulary (OOV) problem.

- **The Concept:** Instead of learning vectors for whole words, FastText learns vectors for **sub-word *n*-grams**.
- **Example:** The word "apple" ($n = 3$) is broken into:
  $< ap, app, ppl, ple, le > + < apple >$
- **Handling OOV:** If a misspelled word like "applz" appears in production, FastText doesn't crash. It looks at the sub-words ($< ap, app, ppl, plz >$), retrieves their individual vectors, and averages them to create a highly accurate guess for the unknown word!

Thank You!

Any Questions?